

EPIC 08 — Task 01: Logging & Distributed Tracing

Your task says to implement:

- Structured logging
- Correlation ID
- Request tracing
- Service identification

Here is the **Step 1 implementation** for your Spring Boot microservice.

1. Add dependencies in pom.xml

```
<!-- Web -->
```

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<!-- Actuator -->
```

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-actuator</artifactId>
```

```
</dependency>
```

```
<!-- Micrometer Tracing -->
```

```
<dependency>
```

```
    <groupId>io.micrometer</groupId>
```

```
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
```

```
</dependency>
```

```
<!-- OpenTelemetry -->
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

After adding them:

Right-click project → Maven → Update Project → Finish

2. Configure the service

Open:

src/main/resources/application.properties

Add:

spring.application.name=order-service

management.endpoints.web.exposure.include=health,info,metrics

management.tracing.sampling.probability=1.0

logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%X{correlationId}] %-
5level %logger{36} - %msg%n

If your service is named differently, replace order-service with your actual service name.

3. Create Correlation ID

In your Java source folder, create:

```
config
└─ CorrelationIdFilter.java
```

Put this code inside:

```
package com.example.orderservice.config;

import java.io.IOException;
import java.util.UUID;

import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {

    private static final String CORRELATION_ID = "correlationId";

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String correlationId =
            request.getHeader(CORRELATION_ID);

        if (correlationId == null || correlationId.isBlank()) {
            correlationId = UUID.randomUUID().toString();
        }

        MDC.put(CORRELATION_ID, correlationId);
        response.setHeader(CORRELATION_ID, correlationId);
    }
}
```

```

    try{
        filterChain.doFilter(request, response);
    } finally{
        MDC.remove(CORRELATION_ID);
    }
}
}

```

Important: Change this line:

```
package com.example.orderservice.config;
```

to match the package name of your project.

4. Add structured logging

In your controller, use a logger:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
@RestController
```

```
public class OrderController {
```

```
    private static final Logger log =
        LoggerFactory.getLogger(OrderController.class);
```

```
    @GetMapping("/orders")
    public String getOrders() {
```

```
        log.info("Fetching orders");
```

```
        return "Orders retrieved successfully";
```

```
    }
}
```

Do **not** use:

```
System.out.println();
```

Use:

```
log.info();  
log.warn();  
log.error();
```

5. Test it

Run your application from Eclipse.

Then open:

<http://localhost:8080/orders>

Check the Eclipse console.

You should see something similar to:

```
2026-09-10 11:50:20 [a82c91f4-...] INFO OrderController - Fetching orders
```

The value:

```
a82c91f4-...
```

Is your **Correlation ID**.

Task 01 result

You have now implemented:

Requirement	Implementation
Structured logging	SLF4J
Correlation ID	CorrelationIdFilter
Request tracing	Micrometer Tracing
Service identification	spring.application.name

This completes Task 01 — Logging & Distributed Tracing.